```
// قفل التطبيق + تشفير نسخة الأوفلاين على الجهاز
// - مفتاح البيانات DK عشوائي، ويبتخزن مغلّف مرتين: مرة بكلمة المرور (PBKDF2) ومرة بمفتاح الجهاز (للبصمة)
// - نسخة الأوفلاين (بيانات عامة بس) متشفرة AES-GCM في IndexedDB
const LS = 'idc.lock', enc = new TextEncoder(), dec = new TextDecoder();
let DK = null;

const b64 = b => { const u = new Uint8Array(b); let s = ''; for (let i = 0; i <
u.length; i++) s += String.fromCharCode(u[i]); return btoa(s); };
const ub = s => Uint8Array.from(atob(s), c => c.charCodeAt(0));
const rnd = n => crypto.getRandomValues(new Uint8Array(n));

const idb = () => new Promise((res, rej) => { const r = indexedDB.open('idc', 1);
r.onupgradeneeded = () => r.result.createObjectStore('kv'); r.onsuccess = () =>
res(r.result); r.onerror = () => rej(r.error); });
const kv = async (m, k, v) => { const d = await idb(); return new Promise((res, rej)
=> { const s = d.transaction('kv', m === 'get' ? 'readonly' :
'readwrite').objectStore('kv'); const q = m === 'get' ? s.get(k) : m === 'set' ?
s.put(v, k) : s.delete(k); q.onsuccess = () => res(q.result); q.onerror = () =>
rej(q.error); }); };

const kek = async (pw, salt) => crypto.subtle.deriveKey({ name: 'PBKDF2', salt,
iterations: 310000, hash: 'SHA-256' },
  await crypto.subtle.importKey('raw', enc.encode(pw), 'PBKDF2', false,
['deriveKey']), { name: 'AES-GCM', length: 256 }, false, ['encrypt', 'decrypt']);
const seal = async (key, data) => { const iv = rnd(12); return { iv: b64(iv), ct:
b64(await crypto.subtle.encrypt({ name: 'AES-GCM', iv }, key, data)) }; };
const open = (key, o) => crypto.subtle.decrypt({ name: 'AES-GCM', iv: ub(o.iv) },
key, ub(o.ct));
const imp = raw => crypto.subtle.importKey('raw', raw, 'AES-GCM', false, ['encrypt',
'decrypt']);
const meta = () => { try { return JSON.parse(localStorage.getItem(LS)); } catch {
return null; } };
const put = m => localStorage.setItem(LS, JSON.stringify(m));

export const has = uid => { const m = meta(); return !!m && m.uid === uid; };
export const unlocked = () => !!DK;
export const lockNow = () => { DK = null; };
export const bioOn = () => !!meta()?.bio;
export const bioSupported = () => !!window.PublicKeyCredential &&
window.isSecureContext;

export async function setup(uid, pw) {
  const raw = rnd(32), salt = rnd(16);
  const dev = await crypto.subtle.generateKey({ name: 'AES-GCM', length: 256 },
false, ['encrypt', 'decrypt']); // non-extractable
  await kv('set', 'dev', dev);
```

```
  put({ uid, salt: b64(salt), pw: await seal(await kek(pw, salt), raw), dv: await
seal(dev, raw), fails: 0 });
  DK = await imp(raw);
}
export async function unlockPw(pw) {
  const m = meta();
  try { DK = await imp(await open(await kek(pw, ub(m.salt)), m.pw)); m.fails = 0;
put(m); return { ok: true }; }
  catch {
    m.fails = (m.fails || 0) + 1;
    if (m.fails >= 5) { await wipeCache(); m.fails = 0; put(m); return { ok: false,
wiped: true }; }
    put(m); return { ok: false, left: 5 - m.fails };
  }
}
export async function changePw(oldPw, newPw) {
  const m = meta(), raw = await open(await kek(oldPw, ub(m.salt)), m.pw), salt =
rnd(16);   // بيرمي error لو القديمة غلط
  m.salt = b64(salt); m.pw = await seal(await kek(newPw, salt), raw); put(m);
}
// البصمة / Face ID عن طريق WebAuthn (المصادقة بتتم من نظام الموبايل نفسه)
export async function enableBio(uid, name) {
  const cred = await navigator.credentials.create({ publicKey: {
    challenge: rnd(32), rp: { name: 'IDC' }, user: { id: enc.encode(uid), name,
displayName: name },
    pubKeyCredParams: [{ type: 'public-key', alg: -7 }, { type: 'public-key', alg:
-257 }],
    authenticatorSelection: { authenticatorAttachment: 'platform', userVerification:
'required' }, timeout: 60000 } });
  const m = meta(); m.bio = b64(cred.rawId); put(m);
}
export async function unlockBio() {
  const m = meta();
  await navigator.credentials.get({ publicKey: { challenge: rnd(32),
userVerification: 'required', timeout: 60000,
    allowCredentials: [{ type: 'public-key', id: ub(m.bio), transports: ['internal']
}] } });
  DK = await imp(await open(await kv('get', 'dev'), m.dv));
}
export async function saveCache(o) { if (DK) await kv('set', 'cache', await seal(DK,
enc.encode(JSON.stringify(o)))); }
export async function loadCache() {
  const c = await kv('get', 'cache'); if (!c || !DK) return null;
  try { return JSON.parse(dec.decode(await open(DK, c))); } catch { return null; }
}
export const wipeCache = () => kv('del', 'cache');
export async function reset() { localStorage.removeItem(LS); await kv('del', 'dev');
await wipeCache(); DK = null; }
```